



241-405 Optimization

Graph Optimization2

The Shortest Path Problem

YOSSAWEE KEAOMANEE



Outline

- Dijkstra
- Floyd-Warshall
- A^*



Objectives

- เข้าใจแนวคิดเกี่ยวกับการหาเส้นทางสั้นที่สุดในกราฟ.
- เรียนรู้และประยุกต์ใช้อัลกอริทึมในการแก้ปัญหาการหาเส้นทางสั้นที่สุด.
- ตระหนักและเข้าใจถึงการนำการหาค่าเหมาะสมที่สุดของเส้นทางสั้นที่สุดไปประยุกต์ใช้.



What is the Shortest Path Problem?

ปัญหาการหาเส้นทางสั้นที่สุด
(Shortest Path Problem) คือ
โจทย์ปัญหาที่เกี่ยวข้องกับการค้นหา
เส้นทางระหว่างจุดยอดสองจุด (หรือ
ระหว่างจุดยอดทุกคู่) ที่ทำให้ผลรวม
ของน้ำหนักบนเส้นเชื่อมในเส้นทาง
นั้นมีค่าน้อยที่สุด





What is the Shortest Path Problem?

ประเภทของปัญหาการหาเส้นทางสั้นที่สุด

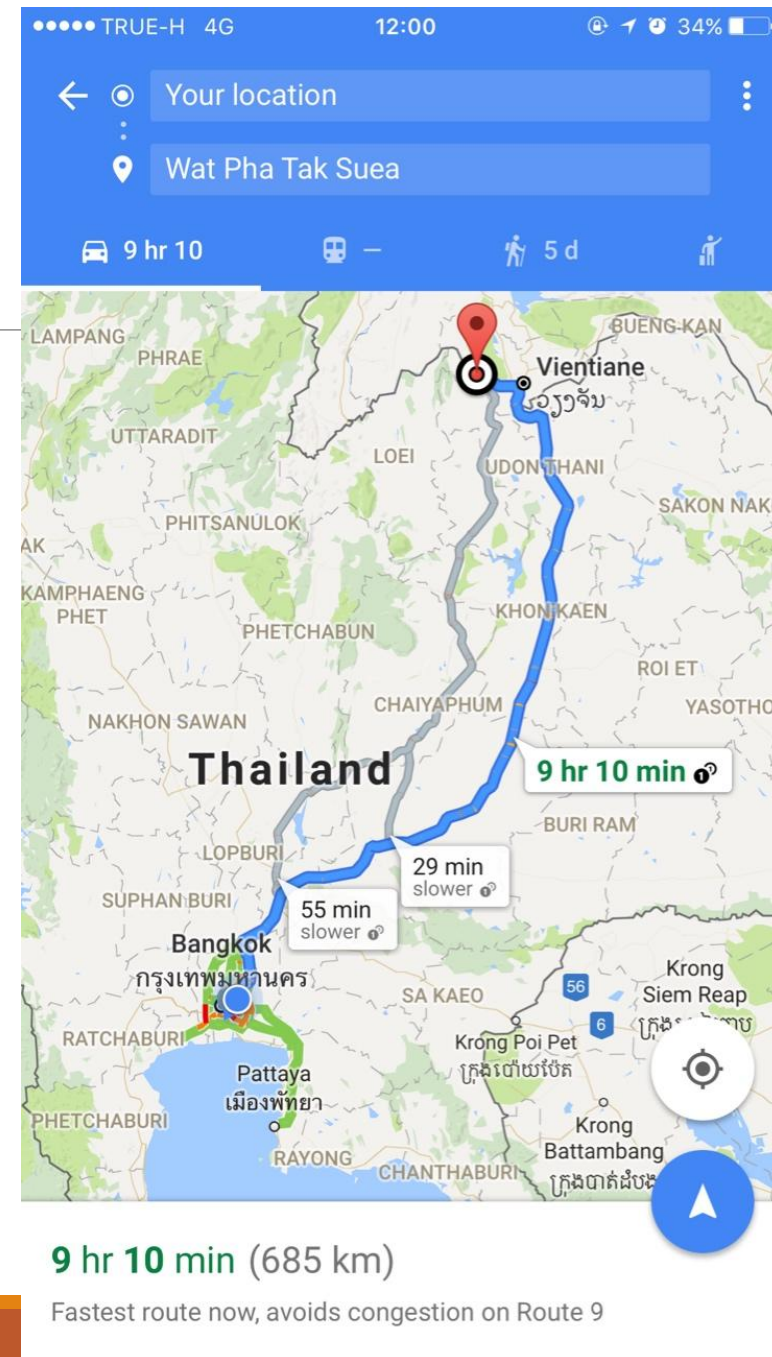
- เส้นทางสั้นที่สุดจากจุดเริ่มต้นเดียว (Single-Source Shortest Path) : ค้นหาเส้นทางที่สั้นที่สุดจากจุดยอดหนึ่งจุด ไปยังจุดยอดอื่น ๆ ทั้งหมด
- เส้นทางสั้นที่สุดระหว่างคู่จุดยอด (Single-Pair Shortest Path) : ค้นหาเส้นทางที่สั้นที่สุด ระหว่างจุดยอดสองจุดที่กำหนด
- เส้นทางสั้นที่สุดระหว่างทุกคู่จุดยอด (All-Pairs Shortest Path) : ค้นหาเส้นทางที่สั้นที่สุด ระหว่างจุดยอดทุกคู่ในกราฟ



Real-world Applications

Google Maps:

การหาเส้นทางที่เร็วที่สุด

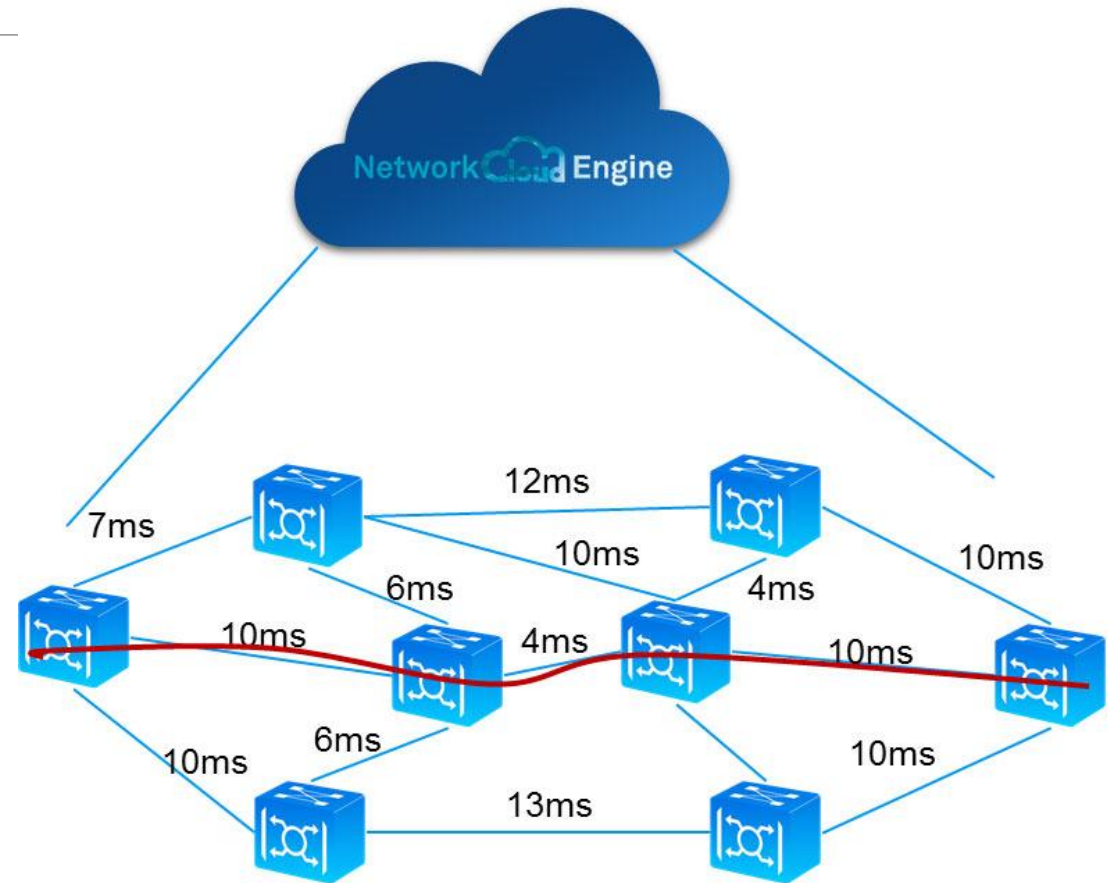




Real-world Applications

การหาเส้นทางในระบบเครือข่าย:

เส้นทางการส่งแพ็กเกจข้อมูลที่มี
ประสิทธิภาพที่สุด

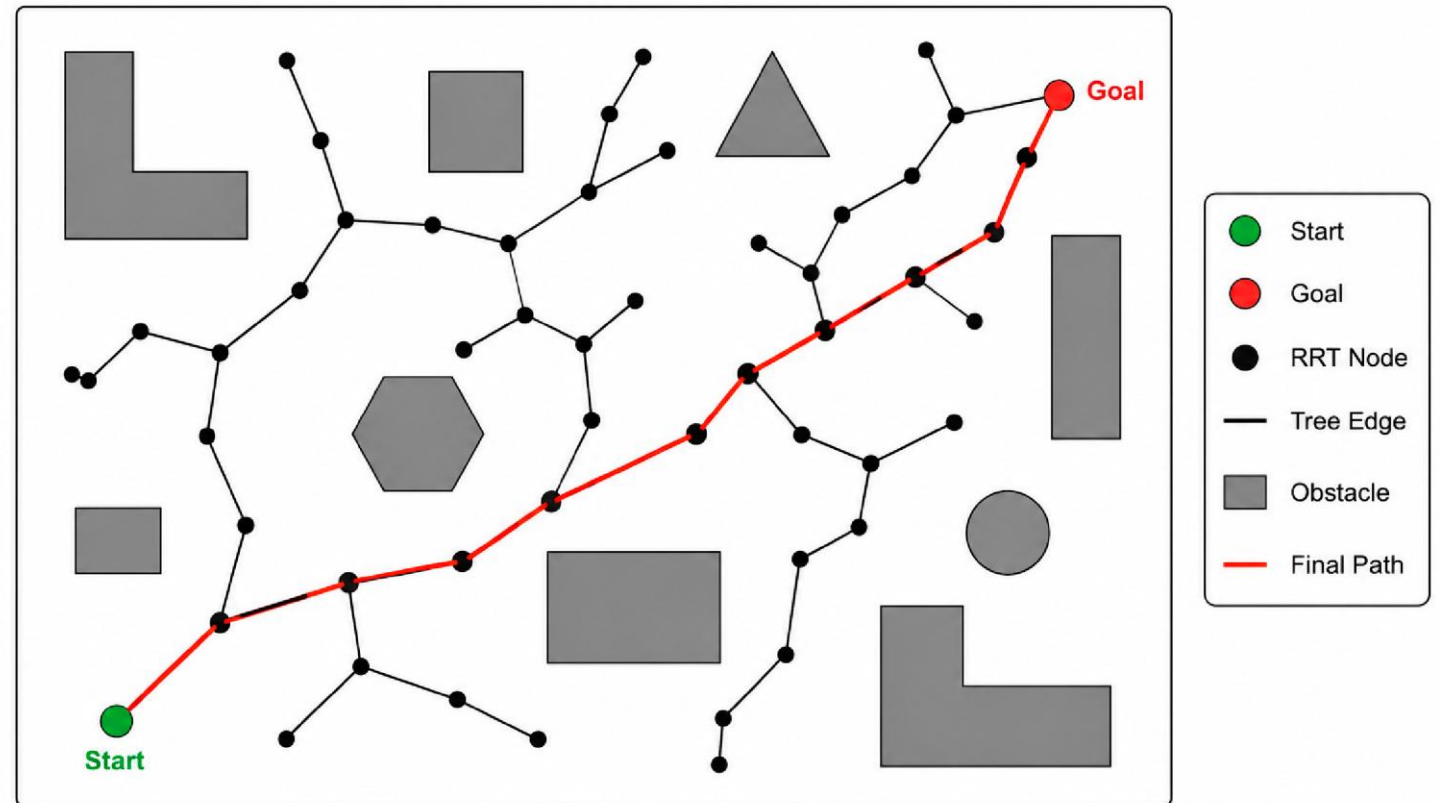




Real-world Applications

วิทยาการหุ่นยนต์:

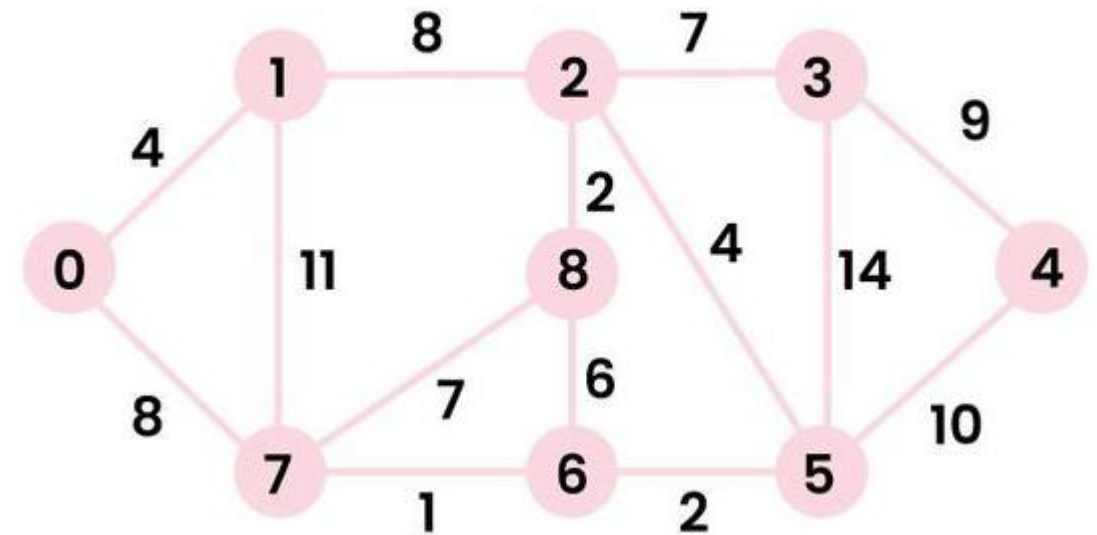
การวางแผนเส้นทางเพื่อการ
นำทางของหุ่นยนต์





Dijkstra's Algorithm

- ใช้สำหรับค้นหาเส้นทางที่สั้นที่สุดจากจุดเริ่มต้น (Source Node) ไปยังจุดยอดอื่นๆ ทั้งหมดภายในกราฟถ่วงน้ำหนัก (Weighted Graph)
- มีเงื่อนไขว่า ค่าน้ำหนักบนเส้นเชื่อมจะต้องไม่เป็นค่าลบ (Non-negative Weights)





Dijkstra's Algorithm

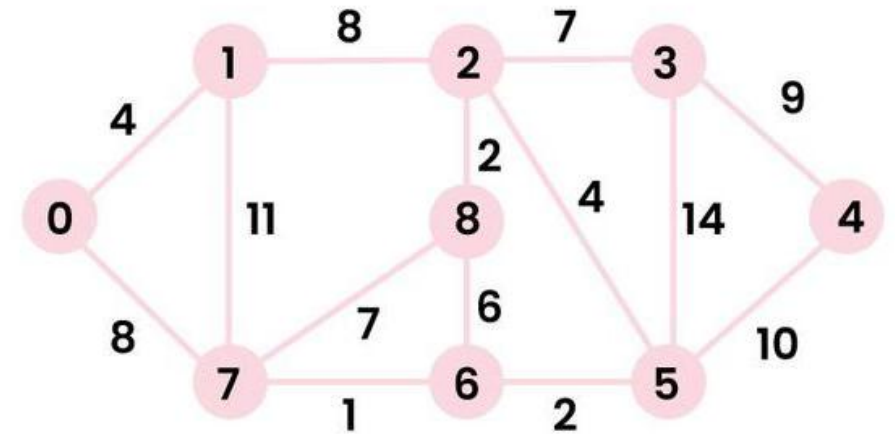
Key Use Case:

ระบบการค้นหาเส้นทางและการนำทาง (Routing/Navigation systems) เช่น Google Maps หรือระบบ GPS

การเพิ่มประสิทธิภาพเครือข่าย (Network optimization) เช่น การหาเส้นทางส่งข้อมูลในระบบอินเทอร์เน็ตที่เร็วที่สุด

การวางแผนเส้นทางของหุ่นยนต์ (Robot path planning) เพื่อให้หุ่นยนต์เคลื่อนที่หลบสิ่งกีดขวางในระยะทางที่สั้นที่สุด

ออกแบบมาสำหรับโจทย์ประเภทจุดเริ่มต้นเดียว (Design for single source) คือการหาเส้นทางจากจุดเริ่มต้นจุดหนึ่งไปยังจุดอื่น ๆ ทุกจุดพร้อมกัน

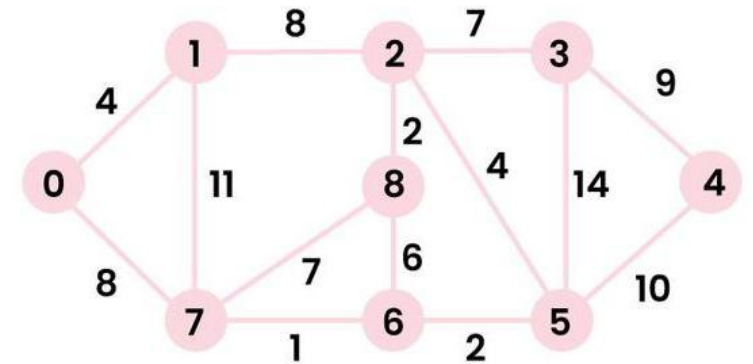




Graph Structure

อัลกอริทึมของ Dijkstra ทำงานบนกราฟแบบถ่วงน้ำหนัก (Weighted Graph) ซึ่งสามารถแทนข้อมูลในหน่วยความจำได้ในรูปแบบดังนี้:

- รายการส่วนประชิด (Adjacency List): มีประสิทธิภาพสูง (ประหยัดพื้นที่และทำงานได้รวดเร็ว)
- เมทริกซ์ส่วนประชิด (Adjacency Matrix): นิยมใช้กับกราฟที่มีขนาดเล็ก





Core Idea of the Algorithm

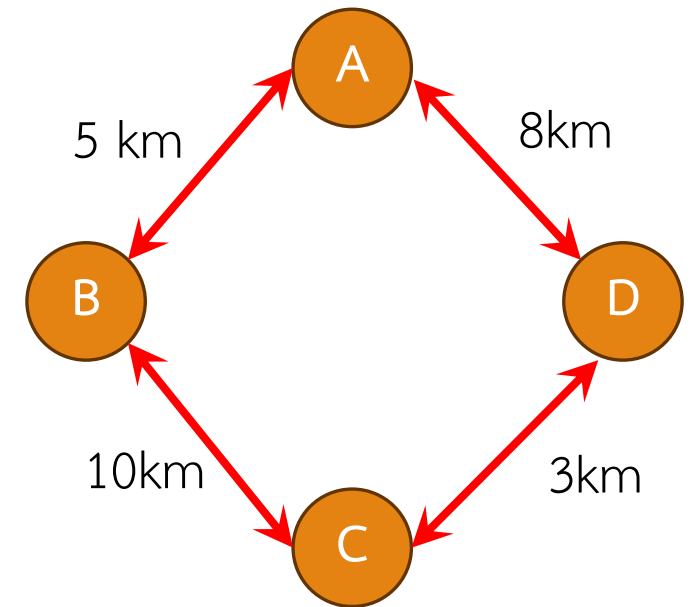
แต่ละโหนดจะมีระยะทางชั่วคราว (กำหนดค่าเริ่มต้นเป็นอินฟินิตี้ ∞ ยกเว้นโหนดเริ่มต้น ซึ่งมีค่าเป็น 0)

ใช้คิวลำดับความสำคัญ (min-heap) เพื่อเลือกโหนดที่มีระยะทางสั้นที่สุดที่ทราบในขณะนั้นมาประมวลผล

สำหรับแต่ละโหนดปัจจุบันที่กำลังประมวลผล:

- ตรวจสอบโหนดข้างเคียงทั้งหมด
- หากพบเส้นทางที่สั้นกว่า ให้ทำการอัปเดตระยะทางใหม่

ทำซ้ำขั้นตอนข้างต้นจนกว่าจะไปเยือนครบทุกโหนด



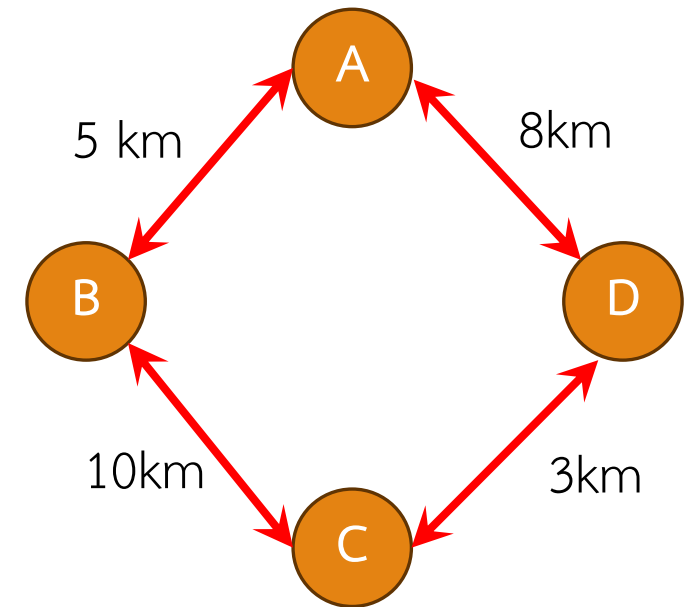


How It Works (Step-by-Step)

หาเส้นทางที่สั้นที่สุดจากจุด A ไปยังจุดอื่น ๆ ทั้งหมด.

ขั้นตอนการตั้งค่าเริ่มต้น (Initialization):

- เริ่มต้นที่จุด A
- ระยะทางไปยังจุด A = 0 ส่วนจุดอื่น ๆ ทั้งหมด = ∞
- จุดก่อนหน้า (Previous node) ของทุกจุด = ยังไม่กำหนด (undefined)





How It Works (Step-by-Step)

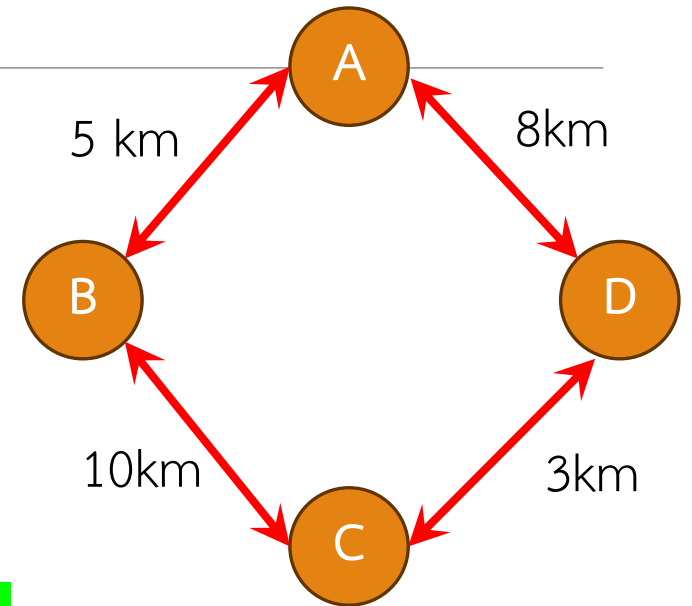
รอบที่ 1 (เยือนโหนด A) :

อัปเดตระยะทางไปยังโหนดข้างเคียง :

- $A \rightarrow B: 0 + 5 = 5$
- $A \rightarrow D: 0 + 8 = 8$

ค่าประมาณระยะทางปัจจุบัน: A: 0, B: 5, C: ∞ , D: 8

ทำเครื่องหมายโหนด A ว่า “มาถึงแล้ว” (Visited).





How It Works (Step-by-Step)

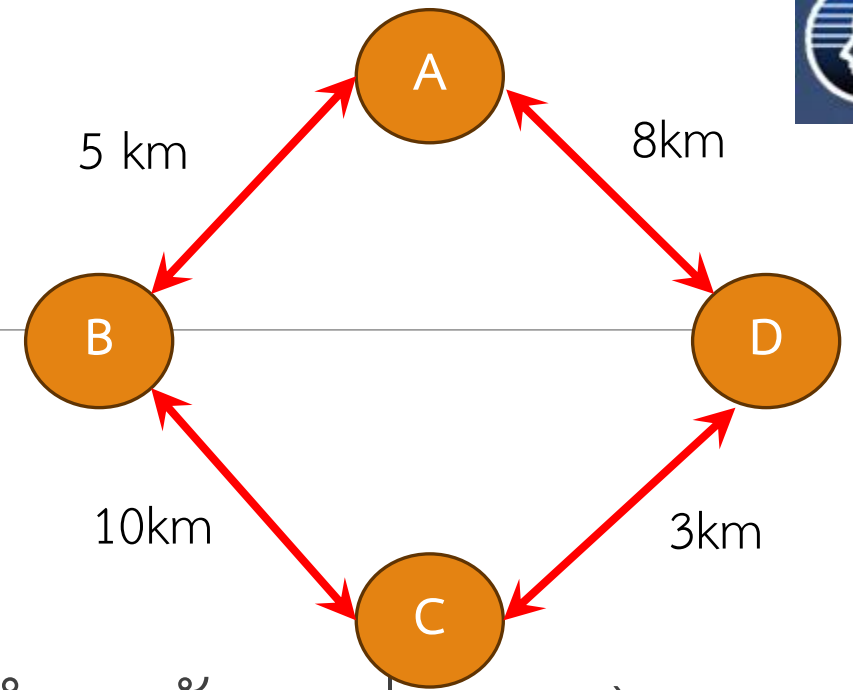
รอบที่ 2 (เยือนโหนด B):

อัปเดตระยะทางจาก B:

- $B \rightarrow C: 5 + 10 = 15$ (เนื่องจาก $15 < \infty$, ทำการอัปเดตค่าของ C)

ค่าประมาณระยะทางล่าสุด : A: 0, B: 5, C: 15, D: 8

ทำเครื่องหมายโหนด B ว่า “มาถึงแล้ว” (Visited).





How It Works (Step-by-Step)

รอบที่ 3 (เยือนโหนด D):

อัปเดตระยะทางจาก D:

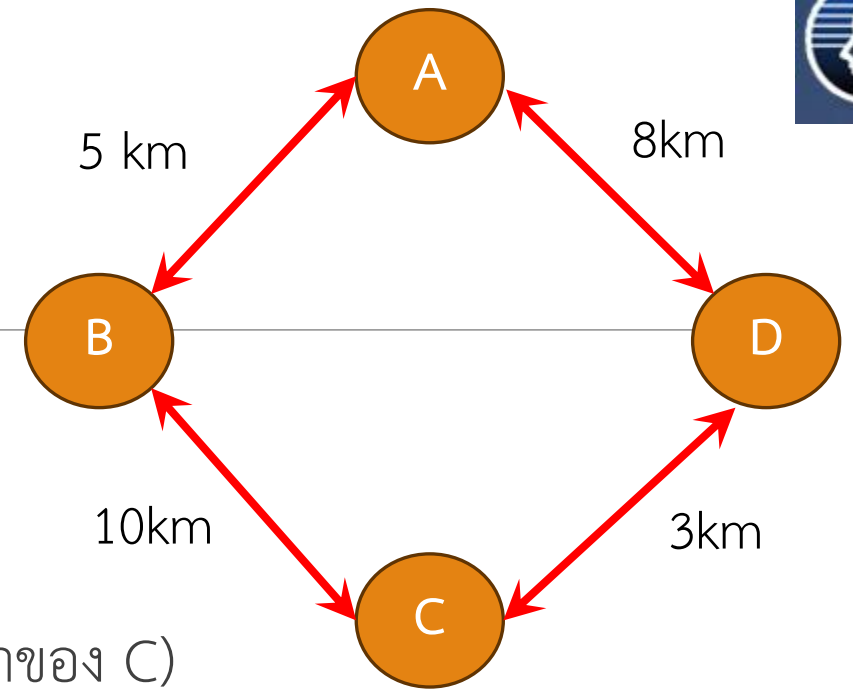
- $D \rightarrow C: 8 + 3 = 11$ (เนื่องจาก $11 < 15$, ทำการอัปเดตค่าของ C)

ค่าประมาณระยะทางล่าสุด : A: 0, B: 5, C: 11, D: 8

ทำเครื่องหมายโหนด D ว่า “มาถึงแล้ว” (Visited).

รอบที่ 4 (เยือนโหนด C):

ไม่มีการอัปเดตข้อมูล ทำเครื่องหมาย C ว่ามาถึงแล้ว





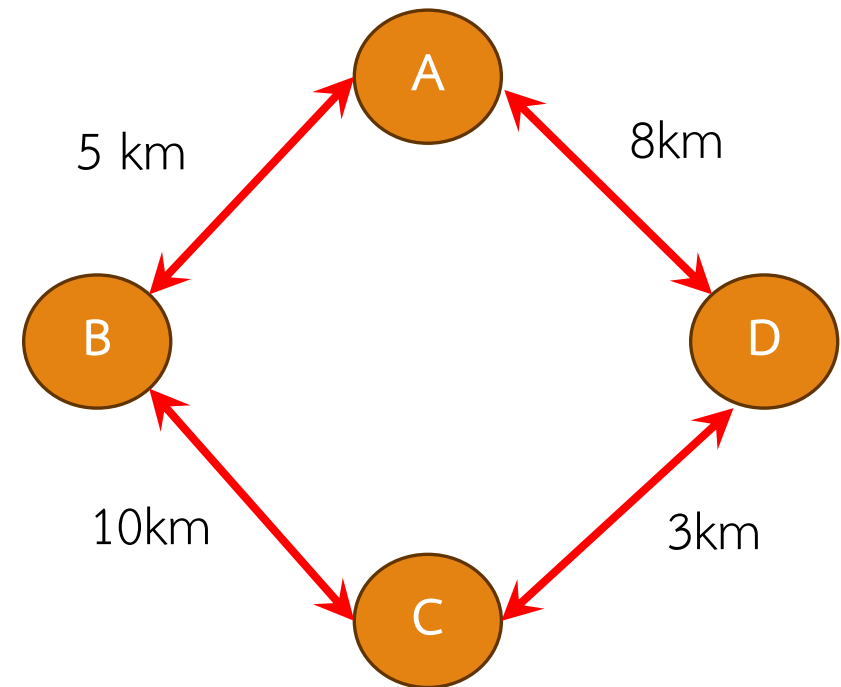
How It Works (Step-by-Step)

สรุปผลระยะทางที่สั้นที่สุดจาก A :

A $\rightarrow$ B: 5 km

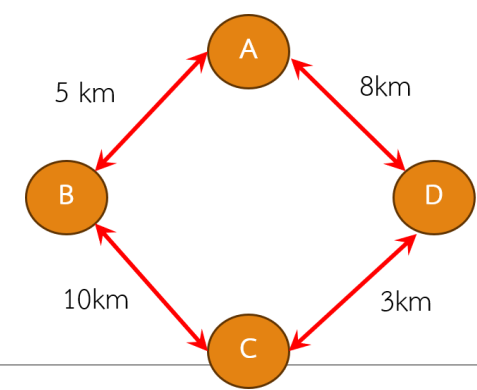
A $\rightarrow$ C: 11 km (via D)

A $\rightarrow$ D: 8 km





How It Works (Step-by-Step)



Step	Current Node	Updated Distances	Priority Queue
0	A	$A=0, B=\infty, C=\infty, D=\infty$	$[(0, A)]$
1	A	$A=0, B=5, D=8$	$[(5, B), (8, D)]$
2	B	$A=0, B=5, C=15$ (via $B \rightarrow C=5+10$), $D=8$	$[(8, D), (15, C)]$
3	D	$A=0, B=5, C=11$ (better via $D \rightarrow C=8+3$)	$[(11, C), (15, C)]$
4	C	All distances finalized	$[]$



How It Works (Step-by-Step)

สรุปผลระยะทางที่สั้นที่สุดจาก A:

Node	Shortest Distance
A	0
B	5
C	11
D	8



Problem 1

วาด graph และหาเส้นทางที่สั้นที่สุดจากจุด A ไปยังจุดอื่น ๆ ทั้งหมด โดยใช้
อัลกอริทึมของ Dijkstra

	A	B	C	D	E	F	G
A	0	4	2	∞	∞	∞	∞
B	4	0	∞	5	10	∞	∞
C	2	∞	0	8	∞	10	∞
D	∞	5	8	0	2	∞	∞
E	∞	10	∞	2	0	∞	3
F	∞	∞	10	∞	∞	0	∞
G	∞	∞	∞	∞	3	∞	0



HW_Problem 1

รอบที่ 1 (สำรวจโหนด A):

อัปเดตระยะทางไปยังโหนดรอบข้าง:

■ $A \rightarrow B: 0 + 4 = 4$

■ $A \rightarrow C: 0 + 2 = 2$

ค่าระยะทางประมาณการ : A:0, B:4, C:2, D:∞, E:∞, F:∞, G:∞

ทำเครื่องหมายว่าโหนด A ถูกสำรวจแล้ว (Visited)

	A	B	C	D	E	F	G
A	0	4	2	∞	∞	∞	∞
B	4	0	∞	5	10	∞	∞
C	2	∞	0	8	∞	10	∞
D	∞	5	8	0	2	∞	∞
E	∞	10	∞	2	0	∞	3
F	∞	∞	10	∞	∞	0	∞
G	∞	∞	∞	∞	3	∞	0

Node	Distance	Prev Node
A	0	-
B	4	A
C	2	A
D	∞	-
E	∞	-
F	∞	-
G	∞	-



HW_Problem 1

รอบที่ 2 (พิจารณาโหนด C):

อัปเดตระยะทางจาก C ไปยังโหนดข้างเคียง:

- $C \rightarrow D: 2 + 8 = 10$ (เนื่องจาก $10 < \infty$ อัปเดตค่าของ D)
- $C \rightarrow F: 2 + 10 = 12$ (เนื่องจาก $12 < \infty$ อัปเดตค่าของ F)

ค่าระยะทางประมาณการ : A:0, B:4, C:2, D:10, F:12, E, G
= ∞

ทำเครื่องหมายว่าโหนด C ถูกเยี่ยมชมแล้ว (Visited)

	A	B	C	D	E	F	G
A	0	4	2	10	∞	12	∞
B	4	0	∞	5	10	∞	∞
C	2	∞	0	8	∞	10	∞
D	10	5	8	0	2	∞	∞
E	∞	10	∞	2	0	∞	3
F	12	∞	10	∞	∞	0	∞
G	∞	∞	∞	∞	3	∞	0

Node	Distance	Prev Node
A	0	-
B	4	A
C	2	A
D	10	A $\rightarrow$ C
E	∞	-
F	12	A $\rightarrow$ C
G	∞	-



HW_Problem 1

รอบที่ 3 (พิจารณาโหนด B): ระยะทางที่สั้นที่สุดจาก A
ยกเว้นโหนด C (ซึ่งผ่านการพิจารณาไปแล้ว)

	A	B	C	D	E	F	G
A	0	4	2	9	14	12	∞
B	4	0	∞	5	10	∞	∞
C	2	∞	0	8	∞	10	∞
D	9	5	8	0	2	∞	∞
E	14	10	∞	2	0	∞	3
F	12	∞	10	∞	∞	0	∞
G	∞	∞	∞	∞	3	∞	0

อัปเดตรักษาและคำนวณระยะทางจาก B:

- $B \rightarrow D: 4 + 5 = 9$ (เนื่องจาก $9 < 10$ อัปเดตค่าของ D เป็น 9)
- $B \rightarrow E: 4 + 10 = 14$ (เนื่องจาก $14 < \infty$ อัปเดตค่าของ E เป็น 14)

ค่าระยะทางประมาณการ : A:0, B:4, C:2, D:9, E:14, F:12,
G: ∞

ทำเครื่องหมายว่าโหนด B ผ่านการพิจารณาแล้ว (Visited)

Node	Distance	Prev Node
A	0	-
B	4	A
C	2	A
D	9	A $\rightarrow$ B
E	14	A $\rightarrow$ B
F	12	A $\rightarrow$ C
G	∞	-



Problem 1

	A	B	C	D	E	F	G
A	0	4	2	9	11	12	∞
B	4	0	∞	5	10	∞	∞
C	2	∞	0	8	∞	10	∞
D	9	5	8	0	2	∞	∞
E	11	10	∞	2	0	∞	3
F	12	∞	10	∞	∞	0	∞
G	∞	∞	∞	∞	3	∞	0

ขั้นตอนที่ 4 (พิจารณาโหนด D): ยกเว้น C และ B (ซึ่งถูกพิจารณาไปแล้ว)

อัปเดตระยะทางจาก D:

- $D \rightarrow B: 9 + 5 = 14$ (ไม่อัปเดต B เนื่องจากระยะเดิมดีกว่า)
- $D \rightarrow C: 9 + 8 = 17$ (ไม่อัปเดต C เนื่องจากระยะเดิมดีกว่า)
- $D \rightarrow E: 9 + 2 = 11$ (อัปเดต E เพราะ $11 < 14$)

ค่าระยะทางประมาณการ : A:0, B:4, C:2, D:9, E:11, F:12, G: ∞

ทำเครื่องหมายว่า D ถูกพิจารณาแล้ว (Visited)

Node	Distance	Prev Node
A	0	-
B	4	A
C	2	A
D	9	A $\rightarrow$ B
E	11	A $\rightarrow$ B $\rightarrow$ D
F	12	A $\rightarrow$ C
G	∞	-



Problem 1

รอบที่ 5 (พิจารณาโหนด E): ยกเว้น B, C และ D (ซึ่งผ่านการพิจารณาแล้ว)

อัปเดตระยะทางจาก E:

- $E \rightarrow G: 11 + 3 = 14$ (เนื่องจาก $14 < \infty$ อัปเดตค่าของ G)

ค่าระยะทางประมาณการ: A:0, B:4, C:2, D:9,

E: 11, F: 12, G:14

ทำเครื่องหมาย E ว่าผ่านการพิจารณาแล้ว (Visited)

	A	B	C	D	E	F	G
A	0	4	2	9	11	12	14
B	4	0	∞	5	10	∞	∞
C	2	∞	0	8	∞	10	∞
D	9	5	8	0	2	∞	∞
E	11	10	∞	2	0	∞	3
F	12	∞	10	∞	∞	0	∞
G	14	∞	∞	∞	3	∞	0

Node	Distance	Prev Node
A	0	-
B	4	A
C	2	A
D	9	A $\rightarrow$ B
E	11	A $\rightarrow$ B $\rightarrow$ D
F	12	A $\rightarrow$ C
G	14	A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E



Problem 1

รอบที่ 6 (พิจารณาโหนด F): ยกเว้นโหนด B, C, D และ E (ที่ถูกพิจารณาไปแล้ว)

เส้นทาง $F \rightarrow C$: ระยะทาง $12 + 10 = 22$ (ไม่อัปเดตค่าของ C เนื่องจากระยะทางเดิมสั้นกว่าอยู่แล้ว)

ไม่มีการอัปเดตระยะทางใหม่จากโหนด F

รอบที่ 7 (พิจารณาโหนด G): ยกเว้นโหนด B, C, D, E และ F (ที่ถูกเยี่ยมชมไปแล้ว)

เส้นทาง $G \rightarrow E$: ระยะทาง $14 + 3 = 17$ (ไม่อัปเดตค่าของ E เนื่องจากระยะทางเดิมสั้นกว่าอยู่แล้ว)

ไม่มีการอัปเดตระยะทางใหม่จากโหนด G

	A	B	C	D	E	F	G
A	0	4	2	9	11	12	14
B	4	0	∞	5	10	∞	∞
C	2	∞	0	8	∞	10	∞
D	9	5	8	0	2	∞	∞
E	11	10	∞	2	0	∞	3
F	12	∞	10	∞	∞	0	∞
G	14	∞	∞	∞	3	∞	0



Problem 1

- A ไป B: ระยะทาง = 4, เส้นทาง = $A \rightarrow B$
- A ไป C: ระยะทาง = 2, เส้นทาง = $A \rightarrow C$
- A ไป D: ระยะทาง = 9, เส้นทาง = $A \rightarrow B \rightarrow D$
- A ไป E: ระยะทาง = 11, เส้นทาง = $A \rightarrow B \rightarrow D \rightarrow E$
- A ไป F: ระยะทาง = 12, เส้นทาง = $A \rightarrow C \rightarrow F$
- A ไป G: ระยะทาง = 14, เส้นทาง = $A \rightarrow B \rightarrow D \rightarrow E \rightarrow G$

	A	B	C	D	E	F	G
A	0	4	2	9	11	12	14
B	4	0	∞	5	10	∞	∞
C	2	∞	0	8	∞	10	∞
D	9	5	8	0	2	∞	∞
E	11	10	∞	2	0	∞	3
F	12	∞	10	∞	∞	0	∞
G	14	∞	∞	∞	3	∞	0



Exercise

หาระยะทางที่สั้นที่สุดและเส้นทางจากจุด C ไปยัง F โดยใช้อัลกอริทึมของ Dijkstra

	A	B	C	D	E	F
A	0	3	5	∞	∞	∞
B	3	0	1	7	∞	∞
C	5	1	0	4	6	∞
D	∞	7	4	0	2	5
E	∞	∞	6	2	0	3
F	∞	∞	∞	5	3	0



Exercise

หาค่าเส้นทางที่สั้นที่สุดจากโหนด S ไปยังโหนด G โดยใช้อัลกอริทึมของ Dijkstra

	A	B	C	D	E	F
A	0	4	6	∞	∞	∞
B	4	0	∞	2	7	∞
C	6	∞	0	1	1	∞
D	∞	2	1	0	∞	6
E	∞	7	1	∞	0	3
F	∞	∞	∞	6	3	0



All-Pairs Shortest Path

อัลกอริทึม ฟลอยด์-วอร์แชล (Floyd-Warshall Algorithm)

การหาเส้นทางสั้นที่สุดของทุกคู่จุดยอด (All-Pairs Shortest Paths): ทำหน้าที่ค้นหาเส้นทางที่สั้นที่สุดระหว่างทุก ๆ คู่ของจุดยอด (Nodes) ทั้งหมดที่มีอยู่ในกราฟ

รองรับน้ำหนักเส้นเชื่อมที่เป็นลบได้: สามารถคำนวณกราฟที่มีค่าน้ำหนักติดลบได้ (แต่ต้องไม่มีวัฏจักรลบ หรือ Negative Cycles)

แนวคิดสำคัญ (Key Idea) สำหรับจุดยอดแต่ละคู่ (i, j) , ให้ตรวจสอบว่า การเดินทางผ่านจุดยอดตัวกลาง k จะช่วยให้ได้เส้นทางที่สั้นกว่าเดิมหรือไม่:

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$



All-Pairs Shortest Path

ขั้นตอนการหาเส้นทางสั้นที่สุดด้วย Floyd-Warshall Floyd-Warshall

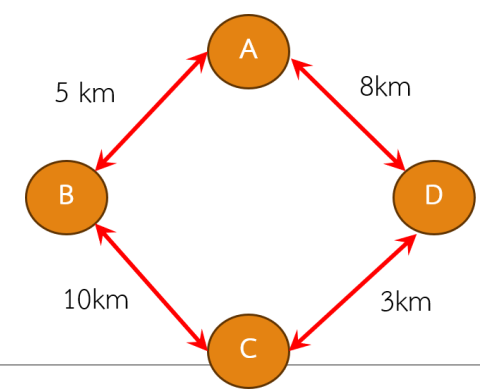
กำหนดเมทริกซ์ระยะทาง D (distance matrix) สำหรับแต่ละโหนด
ตัวกลาง k :

- พิจารณาทุกคู่ของโหนด (i, j) :
- ปรับปรุงค่า $D[i][j]$

หลังจากทำซ้ำครบ n รอบ เมทริกซ์ , $D[i][j]$ จะเก็บค่าระยะทางของ
เส้นทางที่สั้นที่สุดระหว่างทุกคู่โหนด



Floyd-Warshall Algorithm

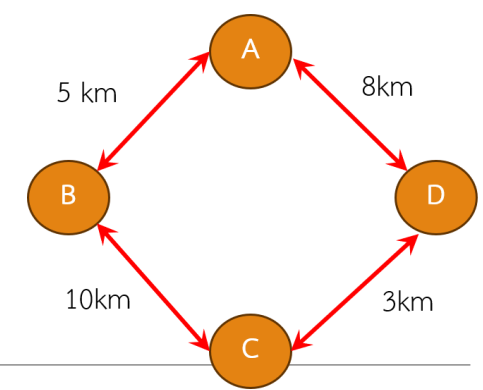


Step 1: กำหนดเมทริกซ์ระยะทาง (distance matrix) D

From/To	A	B	C	D
A	0	5	∞	8
B	5	0	10	∞
C	∞	10	0	3
D	8	∞	3	0

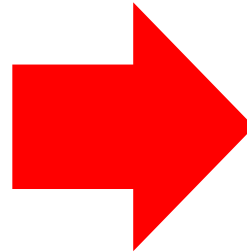


Floyd-Warshall Algorithm



Step 2: (k = A) ตรวจสอบว่าการผ่านจุด A จะช่วยปรับปรุงระยะทางให้สั้นลงหรือไม่.

From/To	A	B	C	D
A	0	5	∞	8
B	5	0	10	∞
C	∞	10	0	3
D	8	∞	3	0



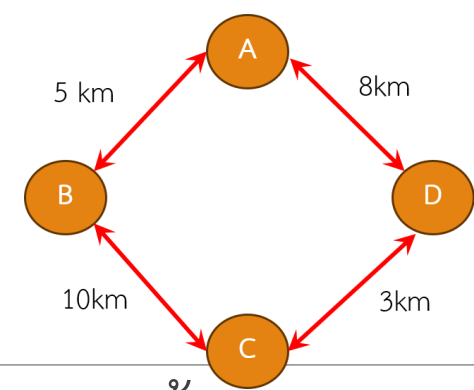
From/To	A	B	C	D
A	0	5	∞	8
B	5	0	10	13
C	∞	10	0	3
D	8	13	3	0

ปรับปรุงค่า $B \rightarrow D$: $B \rightarrow A \rightarrow D = 5 + 8 = 13$

ปรับปรุงค่า $D \rightarrow B$: $D \rightarrow A \rightarrow B = 8 + 5 = 13$

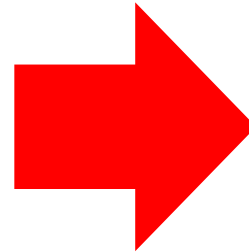


Floyd-Warshall Algorithm



Step 3: (k = B) ตรวจสอบว่าการผ่านจุด B จะช่วยปรับปรุงระยะทางให้สั้นลงหรือไม่.

From/To	A	B	C	D
A	0	5	∞	8
B	5	0	10	13
C	∞	10	0	3
D	8	13	3	0

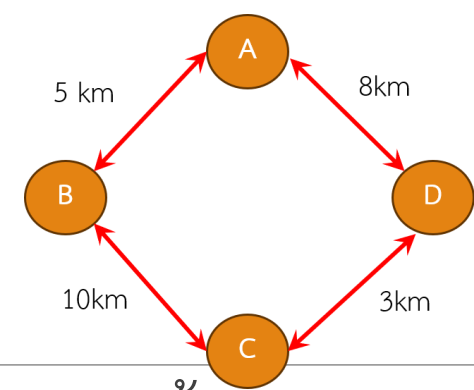


From/To	A	B	C	D
A	0	5	15	8
B	5	0	10	13
C	15	10	0	3
D	8	13	3	0

ปรับปรุงค่า $A \rightarrow C$: $A \rightarrow B \rightarrow C = 5 + 10 = 15$

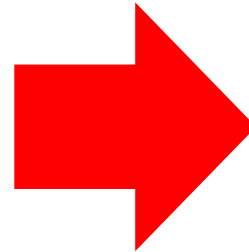


Floyd-Warshall Algorithm



Step 4: ($k = C$) ตรวจสอบว่าการผ่านจุด C จะช่วยปรับปรุงระยะทางให้สั้นลงหรือไม่

From/To	A	B	C	D
A	0	5	15	8
B	5	0	10	13
C	15	10	0	3
D	8	13	3	0

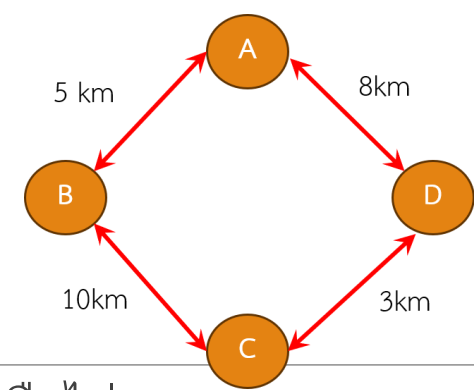


From/To	A	B	C	D
A	0	5	15	8
B	5	0	10	13
C	15	10	0	3
D	8	13	3	0

ไม่มีปรับปรุงค่า.

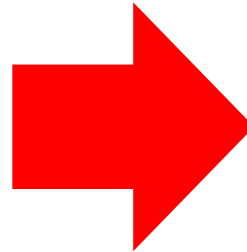


Floyd-Warshall Algorithm



Step 5: (k = D) ตรวจสอบว่าการผ่านจุด D จะช่วยปรับปรุงระยะทางให้สั้นลงหรือไม่.

From/To	A	B	C	D
A	0	5	15	8
B	5	0	10	13
C	15	10	0	3
D	8	13	3	0



From/To	A	B	C	D
A	0	5	11	8
B	5	0	10	13
C	11	10	0	3
D	8	13	3	0

ปรับปรุงค่า $A \rightarrow C$: $A \rightarrow D \rightarrow C = 8 + 3 = 11$

ปรับปรุงค่า $C \rightarrow A$: $C \rightarrow D \rightarrow A = 3 + 8 = 11$



Floyd-Warshall Algorithm

Final Matrix (ระยะทางที่สั้นที่สุด)

From/To	A	B	C	D
A	0	5	11	8
B	5	0	10	13
C	11	10	0	3
D	8	13	3	0



Problem 2

ค้นหาเส้นทางที่สั้นที่สุดโดยใช้วิธี Floyd-Warshall

	A	B	C	D	E	F	G
A	0	4	2	∞	∞	∞	∞
B	4	0	∞	5	10	∞	∞
C	2	∞	0	8	∞	10	∞
D	∞	5	8	0	2	∞	∞
E	∞	10	∞	2	0	∞	3
F	∞	∞	10	∞	∞	0	∞
G	∞	∞	∞	∞	3	∞	0



Problem 2

ขั้นที่ 1: ให้ A เป็นโหนดกลาง ($k = A$)

ใช้คู่ที่ระยะทางสามารถปรับลดได้ด้วยการผ่าน A:

- $B \rightarrow C: \min(\infty, B \rightarrow A + A \rightarrow C) = \min(\infty, 4 + 2) = 6$
- $C \rightarrow B: \min(\infty, C \rightarrow A + A \rightarrow B) = \min(\infty, 2 + 4) = 6$

	A	B	C	D	E	F	G
A	0	4	2	∞	∞	∞	∞
B	4	0	6	5	10	∞	∞
C	2	6	0	8	∞	10	∞
D	∞	5	8	0	2	∞	∞
E	∞	10	∞	2	0	∞	3
F	∞	∞	10	∞	∞	0	∞
G	∞	∞	∞	∞	3	∞	0



Problem 2

ขั้นที่ 2: ให้ B เป็นโหนดกลาง ($k = B$)

อัปเดตเส้นทางที่ผ่าน B:

- $A \rightarrow D: \min(\infty, A \rightarrow B + B \rightarrow D) = \min(\infty, 4 + 5) = 9$
- $A \rightarrow E: \min(\infty, A \rightarrow B + B \rightarrow E) = \min(\infty, 4 + 10) = 14$
- $C \rightarrow D: \min(8, C \rightarrow B + B \rightarrow D) = \min(8, 6 + 5) = 8$ (คงเดิม)
- $C \rightarrow E: \min(\infty, C \rightarrow B + B \rightarrow E) = \min(\infty, 6 + 10) = 16$

	A	B	C	D	E	F	G
A	0	4	2	9	14	∞	∞
B	4	0	6	5	10	∞	∞
C	2	6	0	8	16	10	∞
D	9	5	8	0	2	∞	∞
E	14	10	16	2	0	∞	3
F	∞	∞	10	∞	∞	0	∞
G	∞	∞	∞	∞	3	∞	0



Problem 2

ขั้นที่ 3: ให้ C เป็นโหนดกลาง ($k = C$)

อัปเดตเส้นทางที่ผ่าน C :

- $A \rightarrow F$: $\min(\infty, A \rightarrow C + C \rightarrow F) = \min(\infty, 2 + 10) = 12$
- $B \rightarrow F$: $\min(\infty, B \rightarrow C + C \rightarrow F) = \min(\infty, 6 + 10) = 16$
- $D \rightarrow F$: $\min(\infty, D \rightarrow C + C \rightarrow F) = \min(\infty, 8 + 10) = 18$
- $E \rightarrow F$: $\min(\infty, E \rightarrow C + C \rightarrow F) = \min(\infty, 16 + 10) = 26$

	A	B	C	D	E	F	G
A	0	4	2	9	14	12	∞
B	4	0	6	5	10	16	∞
C	2	6	0	8	16	10	∞
D	9	5	8	0	2	18	∞
E	14	10	16	2	0	26	3
F	12	16	10	18	26	0	∞
G	∞	∞	∞	∞	3	∞	0



Problem 2

ขั้นที่ 4: ให้ D เป็นโหนดกลาง ($k = D$)

อัปเดตเส้นทางที่ผ่าน D :

- $B \rightarrow E$: $\min(10, B \rightarrow D + D \rightarrow E) = \min(10, 5 + 2) = 7$
- $A \rightarrow E$: $\min(14, A \rightarrow D + D \rightarrow E) = \min(14, 9 + 2) = 11$
- $E \rightarrow C$: $\min(\infty, E \rightarrow D + D \rightarrow C) = \min(\infty, 2 + 8) = 10$
- $E \rightarrow F$: $\min(26, E \rightarrow D + D \rightarrow F) = \min(26, 2 + 18) = 20$

	A	B	C	D	E	F	G
A	0	4	2	9	11	12	∞
B	4	0	6	5	7	16	∞
C	2	6	0	8	10	10	∞
D	9	5	8	0	2	18	∞
E	11	7	10	2	0	20	3
F	12	16	10	18	20	0	∞
G	∞	∞	∞	∞	3	∞	0



Problem 2

ขั้นที่ 5: ให้ E เป็นโหนดกลาง ($k = E$)

	A	B	C	D	E	F	G
A	0	4	2	9	11	12	14
B	4	0	6	5	7	16	10
C	2	6	0	8	10	10	13
D	9	5	8	0	2	18	5
E	11	7	10	2	0	20	3
F	12	16	10	18	20	0	23
G	14	10	13	5	3	23	0

$$A \rightarrow G: \min(\infty, A \rightarrow E + E \rightarrow G) = \min(\infty, 11 + 3) = 14$$

$$B \rightarrow G: \min(\infty, B \rightarrow E + E \rightarrow G) = \min(\infty, 7 + 3) = 10$$

$$C \rightarrow G: \min(\infty, C \rightarrow E + E \rightarrow G) = \min(\infty, 10 + 3) = 13$$

$$D \rightarrow G: \min(\infty, D \rightarrow E + E \rightarrow G) = \min(\infty, 2 + 3) = 5$$

$$F \rightarrow G: \min(\infty, F \rightarrow E + E \rightarrow G) = \min(\infty, 20 + 3) = 23$$



Problem 2

ผลลัพธ์

	A	B	C	D	E	F	G
A	0	4	2	9	10	12	13
B	4	0	6	5	7	16	10
C	2	6	0	8	10	10	13
D	9	5	8	0	2	18	5
E	10	7	10	2	0	20	3
F	12	16	10	18	20	0	23
G	13	10	13	5	3	23	0



The A* Algorithm

A* เป็นอัลกอริทึมสำหรับค้นหาเส้นทาง (pathfinding algorithm) ที่ใช้ในการหาเส้นทางที่สั้นที่สุดจากโหนดเริ่มต้น (start node) จุดหนึ่ง ไปยังโหนดเป้าหมาย (goal node) จุดเดียว โดยการผสมผสานระหว่าง:

- หลักการของอัลกอริทึม Dijkstra (Dijkstra's Algorithm) ใช้ในการหาค่าใช้จ่ายจริงจากจุดเริ่มต้นไปยังโหนดปัจจุบัน
- ฮิวริสติก (heuristic) ค่าประมาณของค่าใช้จ่ายจากโหนดปัจจุบันไปยังเป้าหมาย



The A* Algorithm : The key formula

A* จะคำนวณลำดับความสำคัญ (priority) ของแต่ละโหนดโดยใช้ฟังก์ชัน $f(n)$ ดังนี้:

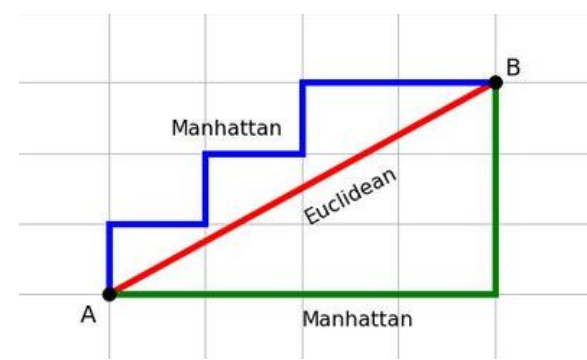
$$f(n) = g(n) + h(n)$$

$g(n)$: ระยะทางจริงจากจุดเริ่มต้นถึงโหนดปัจจุบัน n (สะสมจากน้ำหนักของกราฟ คล้ายกับ Dijkstra)

$h(n)$: ค่าประมาณการระยะทางจากโหนดปัจจุบัน n ไปยังจุดหมาย (Heuristic Function) เช่น ระยะทางเส้นตรง (Euclidean/Manhattan Distance) โดย $h(n)$ ต้องไม่ประเมินค่าเกินจริง (Admissible)



Common heuristics used



Euclidean distance:
$$h(n) = \sqrt{(x_{goal} - x_n)^2 + (y_{goal} - y_n)^2}$$

Manhattan distance:
$$h(n) = |x_{goal} - x_n| + |y_{goal} - y_n|$$

Node	$h(n)$
A	7
B	6
C	2
D	0

Node pair	cost
$A \rightarrow B$	2
$A \rightarrow C$	5
$B \rightarrow C$	1
$C \rightarrow D$	2

	A	B	C	D
A	0	2	5	∞
B	2	0	1	∞
C	5	1	0	2
D	∞	∞	2	0



The A* Algorithm

Step 0: กำหนดนิยาม

$g(n)$ = ต้นทุนจริงจากจุดเริ่มต้น (A) ไปยังโหนด n

$h(n)$ = ค่าประมาณเชิงฮิวริสติกจากโหนด n ไปยังเป้าหมาย

กำหนดฮิวริสติกแบบง่าย (เช่น ระยะทางเส้นตรง) เพื่อใช้เป็นตัวอย่าง



The A* Algorithm

Initialization

- สร้าง Open List (เก็บโหนดที่รอสำรวจ) และใส่โหนดเริ่มต้นลงไป โดยตั้งค่า $g(start) = 0$
- สร้าง Closed List (เก็บโหนดที่สำรวจเสร็จแล้ว)

Loop Selection:

- ดึงโหนด n ที่มีค่า $f(n)$ ต่ำที่สุด จาก Open List ออกมาพิจารณา
- ย้ายโหนด n นั้นเข้า Closed List



The A* Algorithm

Expand Neighbor:

สำหรับแต่ละโหนดเพื่อนบ้าน m ที่เชื่อมต่อกับ n :

- ถ้า m อยู่ใน Closed List แล้ว ให้ข้ามไป
- คำนวณค่า $g_{new} = g(n) + \text{weight}(n, m)$
- ถ้า m ยังไม่อยู่ใน Open List หรือได้ค่า g_{new} ที่สั้นกว่าค่าเดิม:
 - อัปเดต $g(m) = g_{new}$
 - คำนวณ $f(m) = g(m) + h(m)$
 - บันทึกว่า n คือ Parent ของ m และใส่ m ลงใน Open List

The A* Algorithm

Node	$h(n)$		A	B	C	D
A	7	A	0	2	5	∞
B	6	B	2	0	1	∞
C	2	C	5	1	0	2
D	0	D	∞	∞	2	0

Step 1: พิจารณาโหนดเริ่มต้น (A)

อยู่ที่โหนดเริ่มต้น A โดยค่าค่าใช้จ่ายจากจุดเริ่มต้นไปยังตัวเองจะเท่ากับ 0 เสมอ

▪ $g(A) = 0$

ค่า heuristic ของ A คือ $h(A) = 7$ (อ้างอิงจากตาราง)

ดังนั้น ค่าใช้จ่ายโดยประมาณทั้งหมด $f(A)$ สามารถคำนวณได้ดังนี้:

$$f(A) = g(A) + h(A) = 0 + 7 = 7$$

Open List: {A: 7} | Closed List: {}

The A* Algorithm

Node	$h(n)$		A	B	C	D
A	7	A	0	2	5	∞
B	6	B	2	0	1	∞
C	2	C	5	1	0	2
D	0	D	∞	∞	2	0

Step 2: เลือก A (ค่า $f = 7$) นำเข้า Closed List และขยายเพื่อนบ้าน

ไป B: $g(B) = g(A) + cost(A \rightarrow B) = 0 + 2 = 2 \Rightarrow f(B) = 2 + 6 = 8$

ไป C: $g(C) = g(A) + cost(A \rightarrow C) = 0 + 5 = 5 \Rightarrow f(C) = 5 + 2 = 7$

Open List: {C: 7, B: 8} | Closed List: {A}

The A* Algorithm

Node	$h(n)$		A	B	C	D
A	7	A	0	2	5	∞
B	6	B	2	0	1	∞
C	2	C	5	1	0	2
D	0	D	∞	∞	2	0

Step 3: เลือก C (ค่า f น้อยสุด = 7) นำเข้า Closed List และขยายเพื่อนบ้าน

ไป B: $g(B) = g(C) + cost(C \rightarrow B) = 5 + 1 = 6 \Rightarrow f(B) = 6 + 6 = 12$
(เดิมผ่าน $A \rightarrow B$ ได้ $g = 2$ ดีกว่า จึงคงค่าเดิม)

ไป D: $g(D) = g(C) + cost(C \rightarrow D) = 5 + 2 = 7 \Rightarrow f(D) = 7 + 0 = 7$

Open List: {D: 7, B: 8} | Closed List: {A, C}

The A* Algorithm

Node	$h(n)$		A	B	C	D
A	7	A	0	2	5	∞
B	6	B	2	0	1	∞
C	2	C	5	1	0	2
D	0	D	∞	∞	2	0

Step 4: เลือก D (ค่า f น้อยสุด = 7)

โหนดเป้าหมาย D ถูกดึงออกจาก Open List พิจารณาเป็นอันดับถัดไป สิ้นสุดการค้นหาค้นหา

สรุปผลลัพธ์:

เส้นทางที่สั้นที่สุด: $A \rightarrow C \rightarrow D$

ระยะทางจริงรวม (g): $5 + 2 = 7$



Exercise

หาค่าเส้นทางที่สั้นที่สุดจากโหนด S ไปยังโหนด G โดยใช้อัลกอริทึม A* โดยอ้างอิงจากกราฟและตารางค่า heuristic ที่กำหนดไว้ด้านล่าง

Node	$h(n)$
S	10
A	5
B	4
C	3
D	2
G	0

From/to	S	A	B	C	D	G
S	0	4	6	∞	∞	∞
A	4	0	∞	2	7	∞
B	6	∞	0	1	1	∞
C	∞	2	1	0	∞	6
D	∞	7	1	∞	0	3
G	∞	∞	∞	6	3	0



Exercise

Step 1: เริ่มต้นที่โหนด S

$$g(S) = 0$$

$$f(S) = g(S) + h(S) = 0 + 10 = 10$$

Open List: {S: 10} | Closed List: {}

Node	$h(n)$
S	10
A	5
B	4
C	3
D	2
G	0

From/to	S	A	B	C	D	G
S	0	4	6	∞	∞	∞
A	4	0	∞	2	7	∞
B	6	∞	0	1	1	∞
C	∞	2	1	0	∞	6
D	∞	7	1	∞	0	3
G	∞	∞	∞	6	3	0



Exercise

Node	$h(n)$
S	10
A	5
B	4
C	3
D	2
G	0

From/to	S	A	B	C	D	G
S	0	4	6	∞	∞	∞
A	4	0	∞	2	7	∞
B	6	∞	0	1	1	∞
C	∞	2	1	0	∞	6
D	∞	7	1	∞	0	3
G	∞	∞	∞	6	3	0

Step 2: เลือก S($f = 10$) เข้า Closed List แล้วขยายเพื่อนบ้าน

ไป A:

- $g(A) = g(S) + cost(S \rightarrow A) = 0 + 4 = 4$
- $f(A) = g(A) + h(A) = 4 + 5 = 9$

ไป B:

- $g(B) = g(S) + cost(S \rightarrow B) = 0 + 6 = 6$
- $f(B) = g(B) + h(B) = 6 + 4 = 10$

Open List: {A: 9, B: 10} | Closed List: {S}



Exercise

Node	$h(n)$
S	10
A	5
B	4
C	3
D	2
G	0

From/to	S	A	B	C	D	G
S	0	4	6	∞	∞	∞
A	4	0	∞	2	7	∞
B	6	∞	0	1	1	∞
C	∞	2	1	0	∞	6
D	∞	7	1	∞	0	3
G	∞	∞	∞	6	3	0

Step 3: เลือก A ($f = 9$) เข้า Closed List แล้วขยายเพื่อนบ้าน

ไป C:

- $g(C) = g(A) + cost(A \rightarrow C) = 4 + 2 = 6$
- $f(C) = g(C) + h(C) = 6 + 3 = 9$

ไป D:

- $g(D) = g(A) + cost(A \rightarrow D) = 4 + 7 = 11$
- $f(D) = g(D) + h(D) = 11 + 2 = 13$

Open List: {C: 9, B: 10, D: 13} | Closed List: {S, A}



Exercise

Node	$h(n)$
S	10
A	5
B	4
C	3
D	2
G	0

From/to	S	A	B	C	D	G
S	0	4	6	∞	∞	∞
A	4	0	∞	2	7	∞
B	6	∞	0	1	1	∞
C	∞	2	1	0	∞	6
D	∞	7	1	∞	0	3
G	∞	∞	∞	6	3	0

Step 4: เลือก C ($f = 9$) เข้า Closed List แล้วขยายเพื่อนบ้าน

ไป B:

- $g(B)_{new} = g(C) + cost(C \rightarrow B) = 6 + 1 = 7$
- เทียบกับ $g(B)$ เดิมที่มาจาก S ซึ่งมีค่าเท่ากับ 6 $\Rightarrow$ ค่าเดิมสั้นกว่า จึงคง $g(B) = 6$ และ $f(B) = 10$

ไป G:

- $g(G) = g(C) + cost(C \rightarrow G) = 6 + 6 = 12$
- $f(G) = g(G) + h(G) = 12 + 0 = 12$

Open List: {B: 10, G: 12, D: 13} | Closed List: {S, A, C}



Exercise

Node	$h(n)$	From/to	S	A	B	C	D	G
S	10	S	0	4	6	∞	∞	∞
A	5	A	4	0	∞	2	7	∞
B	4	B	6	∞	0	1	1	∞
C	3	C	∞	2	1	0	∞	6
D	2	D	∞	7	1	∞	0	3
G	0	G	∞	∞	∞	6	3	0

Step 5: เลือก B ($f = 10$) เข้า Closed List แล้วขยายเพื่อนบ้าน

ไป D:

- $g(D)_{new} = g(B) + cost(B \rightarrow D) = 6 + 1 = 7$
- เทียบกับ $g(D)$ เดิมผ่าน A ซึ่งมีค่า 11 $\Rightarrow$ ค่าใหม่สั้นกว่า จึงอัปเดตเป็น 7
- $f(D) = g(D)_{new} + h(D) = 7 + 2 = 9$

Open List: {D: 9, G: 12} | Closed List: {S, A, C, B}



Exercise

Node	$h(n)$	From/to	S	A	B	C	D	G
S	10	S	0	4	6	∞	∞	∞
A	5	A	4	0	∞	2	7	∞
B	4	B	6	∞	0	1	1	∞
C	3	C	∞	2	1	0	∞	6
D	2	D	∞	7	1	∞	0	3
G	0	G	∞	∞	∞	6	3	0

Step 6: เลือก D ($f = 9$) เข้า Closed List แล้วขยายเพื่อนบ้าน

ไป G:

- $g(G)_{new} = g(D) + cost(D \rightarrow G) = 7 + 3 = 10$
- เทียบกับ $g(G)$ เดิมผ่าน C ซึ่งมีค่า 12 $\Rightarrow$ ค่าใหม่สั้นกว่า จึงอัปเดตเป็น 10
- $f(G) = g(G)_{new} + h(G) = 10 + 0 = 10$

Open List: {G: 10} | Closed List: {S, A, C, B, D}



Exercise

Step 7: เลือก G ($f = 10$)

Node	$h(n)$	From/to	S	A	B	C	D	G
S	10	S	0	4	6	∞	∞	∞
A	5	A	4	0	∞	2	7	∞
B	4	B	6	∞	0	1	1	∞
C	3	C	∞	2	1	0	∞	6
D	2	D	∞	7	1	∞	0	3
G	0	G	∞	∞	∞	6	3	0

- ดึงโหนดเป้าหมาย G ออกจาก Open List $\Rightarrow$ สิ้นสุดการค้นหา

สรุปผลลัพธ์

- เส้นทางที่สั้นที่สุด: $S \rightarrow B \rightarrow D \rightarrow G$
- ระยะทางจริงรวม (g): $g(G) = g(D) + cost(D \rightarrow G) = 7 + 3 = 10$



Summary

Dijkstra: ใช้ค้นหาเส้นทางที่สั้นที่สุดจากโหนดหนึ่งไปยังทุกโหนด (ปัญหาเส้นทางสั้นที่สุดแบบแหล่งกำเนิดเดียว: Single-Source Shortest Path)

Floyd-Warshall: ใช้ค้นหาเส้นทางที่สั้นที่สุดระหว่างทุกคู่ของโหนด (ปัญหาเส้นทางสั้นที่สุดแบบทุกคู่: All-Pairs Shortest Path)

A*: ใช้ค้นหาเส้นทางที่สั้นที่สุดจากโหนดหนึ่งไปยังเป้าหมายที่กำหนด โดยใช้ heuristics เพื่อช่วยเร่งการค้นหาให้เร็วขึ้น